// BEHIND THE APP · EP 01 · GUIDE

Message Receipts: Sent / Delivered / Read

@thestackexplained
FREE PDF GUIDE



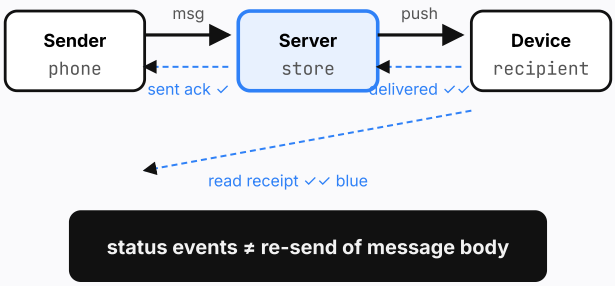
TL;DR Chat ticks are lightweight status events (acks): sent = server stored, delivered = device ack, read = chat opened — not a re-send of the message body.

01 Concept

Modern messaging apps show three receipt states on an outbound message: **sent** (single grey tick), **delivered** (double grey), and **read** (double blue, when read receipts are enabled). These map to an industry-standard ack model, not a claim about any one vendor's private protocol.

Sent: the client hands the message to the messaging server; the server persists it and acks the sender. **Delivered**: the server pushes (or the device fetches) the message; the recipient device acks receipt. **Read**: the recipient client marks the conversation opened and emits a read receipt.

Each transition is a **tiny status event**. The full message body is not re-transmitted for every tick. Think courier tracking: booked → hub → signed — status pings travel without re-shipping the parcel.



Message body flows once; ticks are separate small acks.

02 How it works

1. Client sends the message payload to the messaging server over a persistent connection (e.g. XMPP-style / MQTT / proprietary long-lived socket).
2. Server persists the message (store) and returns a **sent** ack → sender UI shows single grey tick.
3. Server delivers to the recipient device when it is online (push / sync). Device acks → **delivered** (double grey).
4. When the recipient opens the chat (and read receipts are on), the client emits a **read** receipt → double blue ticks on the sender.
5. If the recipient is offline, the server **store-and-forwards**: holds the message until reconnect, then delivers; only then can delivered fire.
6. Groups usually keep **per-participant** delivery/read state; the UI may aggregate (e.g. blue when all have read).

UI	Meaning	Who acks
Single grey ✓	Sent — server accepted & stored	Messaging server
Double grey ✓✓	Delivered — on recipient device	Recipient device
Double blue ✓✓	Read — chat opened (receipts on)	Recipient client
Stays single grey	Recipient offline / not yet delivered	— (pending store-and-forward)
Never turns blue	Read receipts off, or blocked	No read event

03 Real-world implementation

✓ DO THIS IN PRODUCTION

- Model receipts as an append-only **status event stream** keyed by message_id + recipient_id — not by re-writing the message blob.
- Use durable store-and-forward queues per device/session; ack only after local persistence on the device.
- Make status fan-out cheap (pub/sub or push); never poll every message every second.
- Respect privacy: read receipts and last-seen are user settings; server must honour opt-out.
- For groups, store per-participant delivery/read; UI aggregates with clear rules.

```
// Illustrative receipt events (not any vendor's API)
type Receipt = { msgId: string; userId: string; state: 'sent' | 'delivered' | 'read'; at: number };

function onServerStored(msgId: string, senderId: string) {
  emit({ msgId, userId: senderId, state: 'sent', at: Date.now() });
}
function onDeviceAck(msgId: string, recipientId: string) {
  emit({ msgId, userId: recipientId, state: 'delivered', at: Date.now() });
}
function onChatOpened(msgId: string, recipientId: string, receiptsOn: boolean) {
  if (!receiptsOn) return;
  emit({ msgId, userId: recipientId, state: 'read', at: Date.now() });
}
```

⚠ COMMON PITFALLS

- Polling every message every second — O(messages × rate) waste; push status only on change.
- Treating delivered as read — device ack ≠ human opened the chat.
- Ignoring offline: without store-and-forward, messages and ticks both stall.
- Assuming one global read state in groups — each participant has their own.
- Claiming a specific commercial app's private internals without a public source.



04 Interview Q&A

Q. What does a single grey tick usually mean?
The messaging server has accepted and stored the message (sent ack). It has not necessarily reached the recipient device yet.

Q. Why not have the server re-check every message every second?
At crore-message scale, polling burns CPU, bandwidth and battery for no new information. Emit a tiny event only when state actually changes.

Q. When does the delivered tick appear if the recipient is offline?
Only after the device reconnects, the server store-and-forwards the message, and the device acks receipt.

Q. Do blue ticks always mean the person read your message?
They mean the client reported the chat opened while read receipts were enabled — not that a human carefully read every line. Receipts can also be off.

SCENARIO **Q. Scenario: message shows double grey for hours, never blue. Name two likely causes.**
(1) Recipient has read receipts disabled. (2) They received it on device but never opened that chat (or opened in a way that does not emit a read receipt).

Q. How is this different from re-sending the message for each status?
The payload is stored once and delivered once; ticks are separate small ack events referencing the message id.

Q. How do group chats usually handle ticks?
Per-participant delivery/read state is tracked; the UI may show blue only when all (or a defined subset) have read.



// QUICK REVISION

- Sent = server stored
- Delivered = device ack
- Read = chat opened (+ receipts on)
- Status = tiny events, not re-send
- Offline → store-and-forward
- Never poll every msg every second
- Groups = per-participant state